

Software Requirements Specification

ReturnGuard

Autonomous Returns-Deflection & Resolution Agent

An agentic, constrained-autonomy system for e-commerce post-order resolution, built on a stateful agent graph with deterministic financial guardrails, human-in-the-loop escalation, an event-driven pipeline, and an evaluation harness.

Document	Software Requirements Specification
Product (working name)	ReturnGuard
Version	1.0 (implementation baseline)
Author	Akshat Lakhera
Date	June 2026
Standard	ISO/IEC/IEEE 29148:2018 (adapted)

Requirement keywords follow RFC 2119: **MUST/SHALL** (mandatory), **SHOULD** (recommended), **MAY** (optional). All monetary figures are in INR and are configuration-driven illustrative defaults.

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, Acronyms & Abbreviations	4
1.4	References	4
1.5	Document Conventions	4
1.6	Intended Audience	4
2	Overall Description	5
2.1	Product Perspective	5
2.2	Product Functions (Summary)	5
2.3	User Classes and Characteristics	5
2.4	Operating Environment	5
2.5	Design and Implementation Constraints	5
2.6	Assumptions and Dependencies	6
3	System Design & Design Rationale	6
3.1	Design Goals (the forces being optimised)	6
3.2	Chosen Architectural Style	6
3.3	Key Design Decisions	7
3.4	Control-Flow Rationale	8
3.5	Why This Design Is Optimal (for the stated goals)	9
3.6	Trade-offs and Explicitly Deferred Concerns	9
4	System Architecture	9
4.1	High-Level Architecture	9
4.2	Component Decomposition	10
4.3	Agent Orchestration Graph	11
4.4	Data Flow & Control Flow	12
4.5	Event-Driven Pipeline (Kafka)	12
5	Data Requirements	12
5.1	Agent State Schema	12
5.2	Domain Data Model (PostgreSQL)	14
5.3	Policy Knowledge Base (RAG corpus)	14
5.4	Event Schemas (Kafka Contracts)	14
6	External Interface Requirements	15
6.1	User Interfaces	15
6.2	API (FastAPI) Interfaces	15
6.3	MCP Tool Server Interfaces (read)	15
6.4	Action Tool Interfaces (write, simulated)	16
6.5	LLM Provider Interface	16

7	Functional Requirements	16
7.1	Intake & Triage	16
7.2	Context Gathering	16
7.3	Policy Retrieval (RAG)	16
7.4	Fraud & Risk Assessment	17
7.5	Root-Cause Diagnosis	17
7.6	Resolution Planning	17
7.7	Guardrail Enforcement	17
7.8	Human-in-the-Loop Escalation	18
7.9	Action Execution	18
7.10	Customer Response Generation	18
7.11	Outcome Logging & Audit	18
7.12	Event Ingestion & Emission	18
7.13	Analytics & Reporting	19
8	Non-Functional Requirements	19
8.1	Performance & Scalability	19
8.2	Reliability & Durability	19
8.3	Security & Privacy	19
8.4	Safety & Financial Controls	19
8.5	Observability	20
8.6	Maintainability & Extensibility	20
8.7	Compliance & Ethics	20
9	Decision-Logic Specification	20
9.1	Cost Model	20
9.2	Action Catalogue & Eligibility	20
9.3	Decision Rules per Root Cause (default policy)	21
9.4	Constrained-Selection Formulation	21
10	Evaluation & Acceptance Criteria	21
10.1	Evaluation Dataset	22
10.2	Metrics & Targets	22
10.3	Acceptance Criteria	22
10.4	Test Strategy	22
11	Traceability Matrix	23
12	Appendices	23

1 Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for **ReturnGuard**, an autonomous agentic decision system that intercepts e-commerce return, cancellation, and refund requests and resolves them by selecting the lowest-cost action that satisfies platform policy, protects seller margin, and preserves customer satisfaction — escalating to a human operator only when warranted. The system is realised as a stateful agent graph (LangGraph) behind a service layer (FastAPI), driven both synchronously (chat / API) and asynchronously (a Kafka event pipeline). It is intended as a portfolio-grade, demonstrable system that mirrors production patterns used by large commerce and fintech platforms: constrained autonomous action, hard guardrails, human-in-the-loop, observability, and evaluation.

1.2 Scope

Problem. In Indian e-commerce, fashion return rates run 25–35% (up to ~40% in festive periods); COD return-to-origin (RTO) runs 20–40% of COD orders; and sellers lose an estimated 8–15% of monthly revenue to unrecovered return losses. Size/fit mismatch drives 60–70% of fashion returns, and wardrobing accounts for roughly half of all return fraud. Most current handling is a static form plus a human queue; the decision of *whether and how* to deflect, refund, replace, or deny is rule-of-thumb, slow, and unmeasured.

In scope. Classifying inbound post-order issues and diagnosing root cause; retrieving applicable policy via RAG; scoring fraud/abuse risk; selecting an optimal resolution under hard policy/margin/safety constraints; enforcing financial guardrails before any monetary action; escalating high-risk / high-value / policy-ambiguous cases to a human reviewer with a recommendation (pausing and resuming the agent deterministically); executing (simulated) actions (refund, store credit, exchange, retention coupon, expedited replacement, partial refund, goodwill credit, denial, information); generating an empathetic, policy-compliant customer reply; emitting resolution/escalation/audit events; and computing business metrics.

Out of scope. Live integration with any real marketplace, payment gateway, or logistics provider (all external systems are mocked; monetary actions are simulated, never settled); catalogue, checkout, and order-placement flows; pre-purchase RTO prevention at checkout (referenced as future work); production multi-tenant authentication/authorisation, billing, and SRE tooling; and legally exact policy text (policies herein are illustrative).

1.3 Definitions, Acronyms & Abbreviations

Term	Meaning
RTO	Return to Origin — order never reaches the customer and is shipped back to the seller
Return	Customer received the item, then sends it back
Deflection	Resolving an issue without a full reverse-logistics return (exchange, targeted coupon, credit)
Wardrobing	Using an item, then returning it as unused (a fraud pattern)
Bracketing	Ordering multiple sizes/colours intending to return most
COD / LTV	Cash on Delivery / Customer Lifetime Value
HITL	Human-in-the-Loop
RAG / MCP	Retrieval-Augmented Generation / Model Context Protocol
Node / Edge	A unit of computation / a transition (possibly conditional) in the agent graph
State	The typed shared object passed and updated across nodes
Checkpoint	Persistence layer that snapshots graph state for durability and resume
Guardrail	A hard, non-overridable constraint on actions (especially financial)
p95	95th-percentile latency

1.4 References

- LangGraph (orchestration) — github.com/langchain-ai/langgraph; official customer-support tutorial.
- Engineering skeletons — [JoshuaC215/agent-service-toolkit](https://github.com/JoshuaC215/agent-service-toolkit); [wassim249/fastapi-langgraph-agent-production-ready-template](https://github.com/wassim249/fastapi-langgraph-agent-production-ready-template).
- Domain / schema / UI reference — [nitin27may/e-commerce-agents](https://github.com/nitin27may/e-commerce-agents) (orders, return/refund flow, MCP data layer, HITL approval queue; MIT).
- HITL + frontend pattern — [esurovtsev/langgraph-hitl-fastapi-demo](https://github.com/esurovtsev/langgraph-hitl-fastapi-demo).
- Problem-domain figures — IBEF India E-commerce data; TrackVid Ecommerce Return Statistics 2026; Pragma / Ecomdost RTO cost analyses; Rest of World reporting on Indian apparel returns; Product Growth Intelligence returns-as-growth-lever analysis.

1.5 Document Conventions

Requirements use RFC 2119 keywords and are labelled **FR-`<module>-<n>`** (functional), **NFR-`<area>-<n>`** (non-functional), **CON-`<n>`** (constraints), **ASM-`<n>`** (assumptions). Each requirement is intended to be individually testable. Priority tags: [M] mandatory, [S] should, [C] could. All thresholds are configuration-driven (Appendix B).

1.6 Intended Audience

The implementing engineer (primary); technical reviewers evaluating the system; and contributors extending it. Sections 3 (Design Rationale), 9 (Decision Logic), and 10 (Evaluation) are the most load-bearing for demonstrating engineering rigor.

2 Overall Description

2.1 Product Perspective

ReturnGuard sits logically between a commerce platform’s “customer raises a post-order issue” event and the platform’s fulfilment/payment systems. It replaces the implicit human decision step with an auditable, constrained, autonomous agent that defers to humans on the cases that matter. The system is *stateless at the request boundary* (every request is keyed by an idempotent `request_id`) but *stateful within a resolution* (the graph carries typed state, persisted via a durable checkpointer so escalations survive restarts).

2.2 Product Functions (Summary)

The product **SHALL** ingest an issue (chat, API, or Kafka), classify and diagnose it, gather order/customer context, retrieve policy, assess risk, plan a constrained resolution, enforce guardrails, escalate to a human when required, execute the approved (simulated) action, reply to the customer, and record the outcome plus an immutable financial audit entry — while emitting events and exposing business metrics.

2.3 User Classes and Characteristics

User class	Description and interaction	
Customer	Raises the post-order issue in free text; non-technical. Submits a request, receives a reply. Treated as an <i>untrusted</i> input source.	
Operations viewer	Re-	Human who approves/modifies/rejects escalated cases; reviews the queue plus the agent recommendation; the decision resumes the paused graph.
Ops Analyst Manager	/	Monitors performance and cost via the analytics dashboard / metrics API.
Developer / Maintainer	Main-	Builds, configures, extends, evaluates: code, config, eval harness, traces.
Automated producer	Pro-	Upstream system publishing return-request events to the Kafka ingest topic.

2.4 Operating Environment

Python 3.11/3.12; LangGraph 1.x; FastAPI (ASGI/Uvicorn); PostgreSQL 15+ with `pgvector` (domain data, vector index, and the LangGraph checkpointer); Apache Kafka (single broker acceptable); MCP servers (FastMCP, streamable HTTP) for read access; an external chat model accessed by API key (model configurable); LangSmith *or* Langfuse for tracing; an optional Next.js/React operator dashboard; packaged with Docker / docker-compose.

2.5 Design and Implementation Constraints

- **CON-1.** Buildable to a demonstrable state within 2–4 focused days; scope is bounded accordingly.
- **CON-2.** No real money may move and no real PII may be used; data is synthetic and monetary actions are simulated.
- **CON-3.** The graph **MUST NOT** use an in-memory-only checkpointer on any path involving escalation or restart-survival (Postgres checkpointer required).

- **CON-4.** Every conditional/looping path **MUST** have a hard iteration cap guaranteeing termination.
- **CON-5.** Reusable open-source skeletons **SHOULD** provide service, HITL, and observability plumbing; bespoke code concentrates on the graph, tools, decision logic, and evaluation.
- **CON-6.** Customer free-text **MUST** be treated as untrusted data, never as instructions to the agent.

2.6 Assumptions and Dependencies

- **ASM-1.** A synthetic dataset of customers, orders, and labelled issue tickets exists and is produced by a seed script.
- **ASM-2.** An illustrative policy corpus (return rules by category and payment mode) exists in machine-readable form and is embedded for RAG.
- **ASM-3.** External fulfilment/payment/messaging tools are mock implementations with deterministic, inspectable side effects (DB writes + event emission).
- **ASM-4.** An LLM API key and a tracing key are provided via environment variables.
- **ASM-5.** A single operating tenant/seller context is assumed for the demo.

3 System Design & Design Rationale

This section states the design goals the architecture optimises for, the chosen architectural style, and the key decisions with the alternatives that were considered and rejected. The intent is that every significant choice is *defensible*: the design is optimal *relative to the stated goals and the 2–4 day constraint*, not in the absolute, and the costs accepted are named explicitly.

3.1 Design Goals (the forces being optimised)

ID	Goal	Meaning
G1	Constrained autonomy	Act on (simulated) money without a human in the common case, but <i>never</i> unsafely.
G2	Measurable value	Every decision is tied to a rupee cost/saving; success is quantifiable (deflection rate, INR saved).
G3	Trust & auditability	Deterministic guardrails, immutable audit, explainable risk, and HITL for the consequential minority.
G4	Scalability & decoupling	Absorb bursts (festive spikes); integrate with existing systems through events, not tight coupling.
G5	Evaluability	Behaviour pinned by an eval suite with hard pass/fail safety gates.
G6	Demonstrability (≤4 days)	Maximal reuse of proven OSS skeletons; bespoke effort concentrated on the differentiators.
G7	Extensibility	New issue types/actions/policies without re-architecting the graph.

3.2 Chosen Architectural Style

A **stateful agent graph (orchestrated cognition) behind an event-driven service**, wrapped by a **deterministic safety layer with human-in-the-loop**. The problem is in-

trinsically a *multi-step decision* with branching, external lookups, retrieval, and a consequential action — not a single classification or a linear pipeline. A graph with typed shared state and conditional edges models this directly, provides durability/replay/HITL primitives, and keeps each cognitive step small and independently testable (which also controls cost and error relative to a monolithic prompt). The safety layer and HITL exist precisely because the terminal action moves money and therefore cannot depend on model goodwill.

3.3 Key Design Decisions

Each decision below records the chosen option, the alternatives considered, the rationale, and the cost accepted.

D1 — Orchestration framework: *LangGraph*. Considered: a single LLM call; LangChain LCEL chains; LangGraph; CrewAI; AutoGen; a hand-rolled finite-state machine. *Why LangGraph:* it provides typed shared state, conditional/looping edges, durable checkpointing, first-class HITL via interrupt/resume, fan-out, and native tracing — exactly the production primitives this system needs, and it is the de-facto standard for stateful agents (used in production for customer-service and refund flows). A bare LLM call or an LCEL chain cannot express branching + durable pause/resume cleanly; a hand-rolled FSM would re-implement the same primitives with more bugs. *Cost:* a framework dependency and a learning curve, mitigated by reusing OSS skeletons.

D2 — Decision core: *hybrid (LLM perception + deterministic constrained selection)*. Considered: pure rules; pure LLM discretion; a hybrid. *Why hybrid:* pure rules cannot read messy free text or infer nuanced root cause; pure LLM discretion cannot be trusted with unbounded financial authority and cannot *guarantee* policy/margin limits. The hybrid puts the model where ambiguity lives (classification, diagnosis, risk nuance) and puts deterministic logic where money lives (the constrained selection of the action). This also makes the money decision a defensible bounded optimisation (Section 9.4) rather than opaque model output. *Cost:* more moving parts and the need to keep rules and model in sync, mitigated by configuration and the eval suite.

D3 — Ingestion: *event-driven (Kafka) primary, synchronous API secondary*. Considered: synchronous API only; an event-driven pipeline. *Why Kafka:* returns arrive as a high-volume, bursty stream (festive spikes); decoupling producers from the agent through a log gives backpressure, horizontal consumer scaling, replay, and clean integration with existing order systems — and it demonstrates real distributed-systems competence. The synchronous API is retained for chat, testing, and demos. *Cost:* operational complexity (broker, idempotency, dead-letters), bounded by idempotency on `request_id`.

D4 — Data access: *MCP tool servers (read)*. Considered: direct DB calls inside nodes; MCP servers. *Why MCP:* it cleanly separates data access from cognition, the servers are reusable by *any* MCP client (so the data layer becomes a standalone asset), and it aligns with the emerging tool-interoperability standard. *Cost:* an extra process and a layer of indirection — acceptable, and beneficial for testability.

D5 — Policy handling: *RAG over a policy corpus with metadata filtering*. Considered: hard-coding policy into prompts; RAG. *Why RAG:* policies are numerous, vary by category/payment/time, and must be *cited*; embedding them keeps prompts small, makes a

policy update a data change (not a code change), and yields traceable citations for audit and customer replies. *Cost*: retrieval-quality risk, mitigated by metadata filtering (not pure semantic similarity) and by evals.

D6 — Safety: *deterministic guardrails* + *HITL*. Considered: trusting the model to respect limits; a deterministic guardrail layer plus human escalation. *Why guardrails+HITL*: financial safety cannot depend on model compliance — hard caps, policy compliance, and idempotency must be code-enforced; consequential or ambiguous cases need a human, and LangGraph’s interrupt/resume makes that lossless and durable. *Cost*: some cases a bolder system would auto-resolve get escalated — the correct trade for trust.

D7 — Durability: *Postgres checkpoint*. Considered: an in-memory checkpoint; Postgres. *Why Postgres*: escalations can pause for minutes or hours, so state must survive restarts and crashes; in-memory loses active conversations on restart. *Cost*: a DB dependency the system already has.

D8 — Vector store: *pgvector inside the same PostgreSQL*. Considered: a separate vector DB (Pinecone/Chroma/Weaviate); pgvector. *Why pgvector*: one datastore for domain data, vectors, and checkpoints simplifies operations, transactions, and the short build, and is more than sufficient at this scale. *Cost*: a dedicated vector DB would scale embeddings further — unnecessary here.

D9 — Observability: *a tracing platform (LangSmith / Langfuse)*. Considered: print/logs; a tracing platform. *Why tracing*: debugging an agent requires seeing the full trajectory (every node, LLM call, tool call); traces are also the substrate for evaluation and a clear maturity signal. *Cost*: an external dependency/key, kept behind an optional toggle.

D10 — Topology: *a single graph with specialised nodes (not a multi-agent fleet)*. Considered: a fleet of specialised agents communicating via an agent-to-agent protocol; a single well-structured graph. *Why single graph*: the job fits one coherent state and context. Agent-to-agent coordination would add shared-state contracts, new failure surfaces, and debugging cost for no benefit at this scope; multi-agent is justified only when the problem genuinely splits into independent jobs. *Cost*: less buzzword surface — the correct engineering choice.

3.4 Control-Flow Rationale

The graph has exactly *two* branch points: (i) a single-question clarification loop at intake, and (ii) the guardrail→HITL fork before execution. Everything else is linear and deterministic. This keeps the graph legible and testable while still covering the real decision structure; needless branching would multiply states and test surface without modelling anything real.

3.5 Why This Design Is Optimal (for the stated goals)

Goal	Mechanism that satisfies it
G1	Hybrid decision core (D2) executes autonomously but only within the deterministic guardrail envelope (D6).
G2	The cost model (Section 9.1) attaches a rupee figure to every action; metrics (FR-RPT-1) report deflection and INR saved.
G3	Non-overridable guardrails + append-only audit + explainable risk factors + HITL (D6, FR-LOG-2, FR-RSK-2).
G4	Kafka ingestion + stateless workers + pgvector-in-Postgres (D3, D8, NFR-PERF-2).
G5	A labelled eval suite with hard safety gates (Section 10), wired into CI.
G6	OSS skeletons for service/HITL/observability (D1, CON-5); effort spent on graph, decision logic, and evals.
G7	Enum catalogues + config-driven thresholds + drop-in tools (NFR-MNT-1..3).

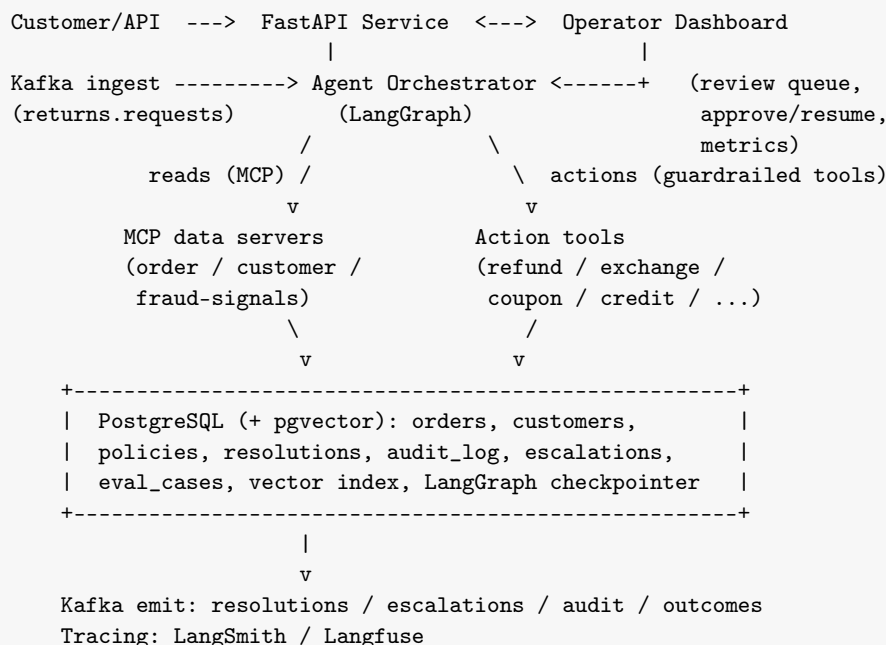
Honest scoping of the optimality claim: “optimal” here means Pareto-best under the goals above and the time constraint. The costs of each decision (D1–D10) and the deferred concerns below are stated so the claim is bounded, not absolute.

3.6 Trade-offs and Explicitly Deferred Concerns

Deliberately deferred to protect the core’s quality and the timeline, with clean architectural seams left for each: pre-purchase RTO prevention (advance-payment nudges, address scoring at checkout); online learning of coupon sizing from realised outcomes; multilingual/vernacular replies; multimodal defect verification (image evidence via a CV model); and real payment/logistics integrations.

4 System Architecture

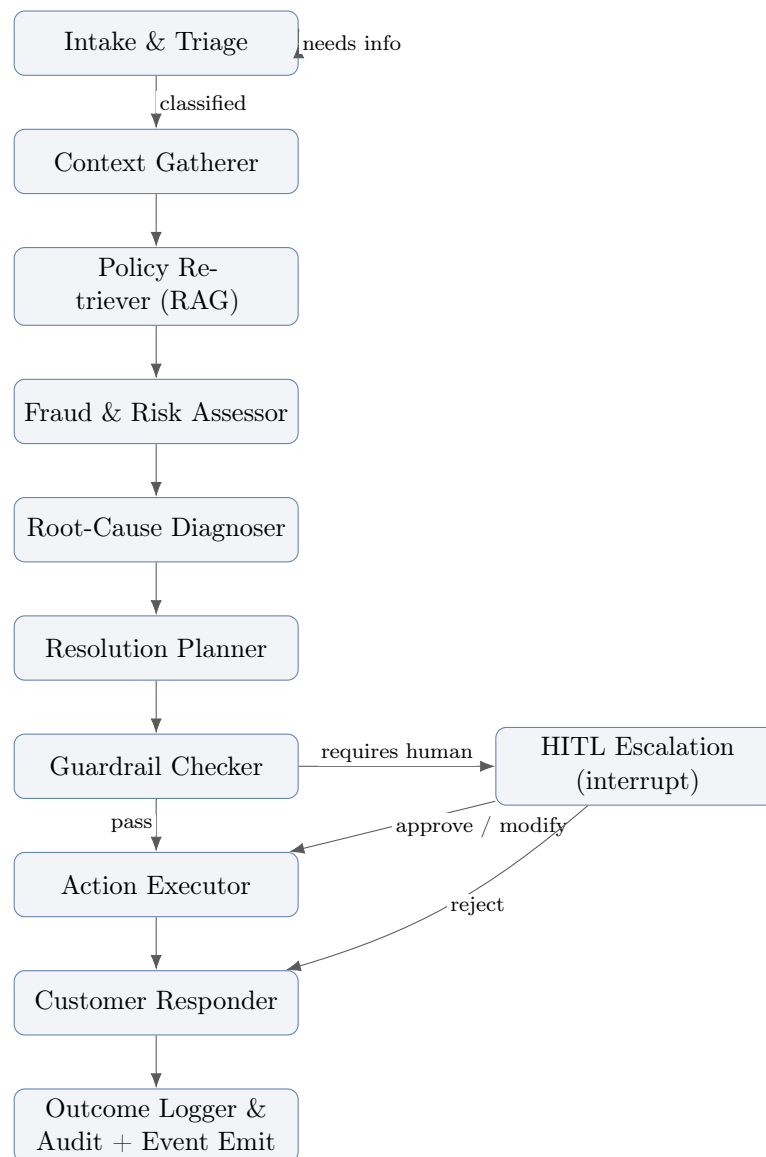
4.1 High-Level Architecture



4.2 Component Decomposition

1. **Service layer (FastAPI):** synchronous resolve endpoints (incl. SSE streaming), the escalation-decision endpoint (resume), queue/resolution/metrics read endpoints, health/readiness.
2. **Event worker:** a Kafka consumer that converts ingest messages into idempotent graph invocations, and a producer that emits resolution/escalation/audit/outcome events.
3. **Agent orchestrator (LangGraph):** the typed `StateGraph` of nodes/edges (Section 4.3, Section 7).
4. **Data access (MCP servers):** read-only order, customer, and fraud-signal lookups.
5. **Action tools:** write/side-effecting (simulated) tools invoked only after guardrails and (when required) human approval.
6. **Knowledge base (RAG):** embedded policy corpus in pgvector with metadata-filtered retrieval.
7. **Persistence:** the Postgres domain schema plus the checkpointer.
8. **Observability:** a trace exporter wrapping every node, LLM call, and tool call.
9. **Operator dashboard (optional):** review queue, decision UI, and metrics.

4.3 Agent Orchestration Graph



Node summary (full requirements in Section 7):

Node	Type	Writes to state
Intake & Triage	LLM + router	issue_type, clarification_needed, extracted ids
Context Gatherer	tool-calling	order_context, customer_context
Policy Retriever	RAG	policy_snippets
Fraud & Risk Assessment	rules + LLM	risk_score, risk_factors
Root-Cause Diagnoser	LLM	root_cause
Resolution Planner	LLM + logic	candidate_actions, proposed_action
Guardrail Checker	deterministic	guardrail_status, requires_human
HITL Escalation	interrupt	human_decision
Action Executor	tool-calling	executed_action, outcome
Customer Responder	LLM	customer_message
Outcome & Audit	deterministic	persisted resolution + audit + events

4.4 Data Flow & Control Flow

Control flows top-to-bottom with the two branch points of Section 3.5. Data accumulates into the shared typed state; nodes **MUST** return only the keys they change (never the whole state), and the `messages` channel **MUST** use an append reducer. State is checkpointed after each node so an escalation can be resumed exactly where it paused.

4.5 Event-Driven Pipeline (Kafka)

A producer publishes a return request to the ingest topic; one of N consumer workers picks it up, invokes the graph, and publishes the resolution. Idempotency is enforced by `request_id`; redelivery **MUST NOT** double-execute a financial action (Section 8.4). Topics and schemas are specified in Section 5.4.

5 Data Requirements

5.1 Agent State Schema

The graph state is a typed structure (TypedDict/Pydantic) and **MUST** include at least:

Field	Type	Notes
messages	list (append)	conversation/tool messages
request_id	str	idempotency key
channel	enum	chat / api / kafka_event
raw_request	str	untrusted customer text
order_id,	cus- tomer_id	extracted or provided
tommer_id		
issue_type	enum / null	see catalogue
clarification_- needed	bool	drives the triage loop
order_context	object / null	fetches order facts
customer_context	object / null	fetches customer facts
policy_snippets	list[object]	retrieved policy + citations
risk_score	float [0,1] / null	abuse/fraud likelihood
risk_factors	list[str]	explainability
root_cause	enum / null	see catalogue
candidate_actions	list[object]	ranked options with cost
proposed_action	object / null	selected pre-guardrail
guardrail_status	enum / null	pass / clamped / violation
requires_human	bool	escalation flag
human_decision	enum / null	approve / modify / reject
executed_action	object / null	final applied action
customer_message	str / null	reply text
outcome	object / null	status, amounts, INR-saved estimate
trace_id	str	observability correlation
iteration_count	int	loop guard

Issue-type catalogue: return_request, cancel_request, refund_status, damaged_item, wrong_item, wrong_size, late_delivery, missing_item, quality_complaint, rto_predicted, other.

Root-cause catalogue: size_fit_mismatch, defect_damage, changed_mind, found_cheaper, delivery_delay, wrong_item_shipped, expectation_mismatch, fraud_suspected, genuine_other.

Action-type catalogue: instant_refund, store_credit_refund, partial_refund, free_exchange, exchange_with_size_guide, retention_coupon, expedited_replacement, goodwill_credit, deny_with_explanation, escalate_to_human, provide_information.

5.2 Domain Data Model (PostgreSQL)

Table	Key columns
customers	id, name, signup_date, segment, ltv, total_orders, total_returns, return_rate, cod_orders, cod_refusals, risk_flags (jsonb), region, pincode
orders	id, customer_id (fk), seller_id, sku, title, category, price, qty, payment_mode {COD, PREPAID}, order_date, dispatch_date, delivery_date, delivery_status, return_window_end
policies	id, category, payment_mode, rule_type, window_days, refundable, exchange_allowed, refund_mode_default, text
policy_chunks	id, policy_id (fk), chunk_text, embedding (vector), metadata (jsonb: category, payment_mode, issue_type)
resolutions	request_id (pk), order_id, customer_id, issue_type, root_cause, risk_score, risk_factors (jsonb), proposed_action (jsonb), executed_action (jsonb), amount, expected_return_cost, expected_saving, requires_human, human_decision, rationale, status, created_at, resolved_at
audit_log	id (pk), request_id (fk), action_type, amount, actor {agent, human:id}, payload (jsonb), created_at — <i>append-only</i>
escalations	request_id (pk/fk), status {pending, decided}, recommendation (jsonb), assigned_to, decided_at, decision
eval_cases	id, scenario_text, seeded_order_id, seeded_customer_id, expected_root_cause, expected_action, expected_escalation, notes

Referential integrity **MUST** be enforced via foreign keys. **audit_log** **MUST** be insert-only at the application layer.

5.3 Policy Knowledge Base (RAG corpus)

- **DR-RAG-1.** The corpus **MUST** be chunked, embedded, and stored in `policy_chunks` with metadata for category, payment_mode, and applicable issue_type.
- **DR-RAG-2.** Retrieval **MUST** apply metadata filtering (category + payment_mode) in addition to semantic similarity, returning top-*k* (default *k* = 4).
- **DR-RAG-3.** Retrieved chunks **MUST** be carried in state with source identifiers so the reply and audit can cite the governing rule.
- **DR-RAG-4.** The corpus **MUST** cover at least: return windows by category; non-returnable categories; defect/damage exceptions; COD vs prepaid refund modes; exchange eligibility; and the principle that a bank/original-method refund is offered by default (store credit only as an incentivised alternative, never the sole option for a genuine refund).

5.4 Event Schemas (Kafka Contracts)

All messages are JSON, versioned by topic suffix, keyed by `request_id`.

Topic `returns.requests.v1` (**ingest**):

```
{ "request_id": "str", "order_id": "str|null", "customer_id": "str|null",
  "issue_text": "str", "issue_type_hint": "str|null",
  "source": "str", "timestamp": "ISO-8601" }
```

Topic `returns.resolutions.v1` (**emit**):

```
{ "request_id": "str", "order_id": "str", "customer_id": "str",
  "issue_type": "str", "root_cause": "str", "risk_score": 0.0,
  "action_type": "str", "amount": 0.0, "requires_human": false,
  "rationale": "str", "expected_saving": 0.0, "status": "str",
  "timestamp": "ISO-8601" }
```

`returns.escalations.v1` (emit): resolution payload + a `recommendation` object, for HITL cases. `returns.audit.v1` (emit): a mirror of each `audit_log` insert. `returns.outcomes.v1` (emit, optional): post-resolution status updates for analytics.

- **DR-EVT-1.** Producers/consumers **MUST** validate messages against these schemas and route invalid messages to a dead-letter path (log + skip; no crash).

6 External Interface Requirements

6.1 User Interfaces

- **UI-1.** A minimal chat/test surface (Swagger UI acceptable) **MUST** allow submitting an issue and viewing the reply + final state.
- **UI-2.** The operator dashboard (**SHOULD**, stretch) **MUST** display the pending escalation queue with the agent recommendation and provide approve/modify/reject controls, and **MUST** display the Section 7.13 metrics.

6.2 API (FastAPI) Interfaces

Endpoint	Method	Purpose
<code>/resolve</code>	POST	Run the graph synchronously; return reply + state summary
<code>/resolve/stream</code>	POST	Same, streamed via SSE (node-by-node)
<code>/escalations</code>	GET	List pending escalations
<code>/escalations/{id}/decision</code>	POST	Submit decision; <i>resumes</i> the paused graph
<code>/resolutions</code>	GET	List/filter resolutions
<code>/resolutions/{id}</code>	GET	Full detail incl. rationale + trace link
<code>/metrics/summary</code>	GET	Aggregate business metrics
<code>/health, /ready</code>	GET	Liveness/readiness

- **API-1.** All write/decision endpoints **MUST** validate inputs (Pydantic) and be idempotent w.r.t. `request_id`.
- **API-2.** The decision endpoint **MUST** reject a decision for a `request_id` not currently paused/awaiting-human.

6.3 MCP Tool Server Interfaces (read)

order-service: `get_order(order_id)`, `get_order_status(order_id)`. customer-service: `get_customer(customer_id)`, `get_return_history(customer_id)`. fraud-service: `get_risk_signals(customer_id, order_id)`.

- **MCP-1.** MCP servers **MUST** be read-only and independently runnable so any MCP client could consume them.

6.4 Action Tool Interfaces (write, simulated)

`process_refund`, `issue_store_credit`, `book_exchange`, `issue_retention_coupon`, `expedite_replacement`, `issue_goodwill_credit`, `create_ticket`, `send_customer_message`.

- **TOOL-1.** Every action tool **MUST** perform its (simulated) effect as a DB write + an `audit_log` insert + an event emission, and return a structured result.
- **TOOL-2.** Action tools **MUST** be invocable only from the Executor node, only after guardrail pass and (if required) human approval.

6.5 LLM Provider Interface

- **LLM-1.** The model **MUST** be configurable (provider, model name) via environment/config.
- **LLM-2.** Every LLM call **MUST** be wrapped with retry-with-backoff and a per-call timeout; structured-output parsing **MUST** be validated, with a safe fallback (route to escalation) on parse failure.

7 Functional Requirements

7.1 Intake & Triage

- **FR-TRI-1** [M]. The system **SHALL** classify `raw_request` into exactly one `issue_type`.
- **FR-TRI-2** [M]. The system **SHALL** extract `order_id` and/or `customer_id` when present.
- **FR-TRI-3** [M]. On missing essential facts, the system **SHALL** set `clarification_needed=true` and ask *at most one* clarifying question per turn, bounded by the iteration cap.
- **FR-TRI-4** [M]. The system **SHALL** treat `raw_request` strictly as data; embedded instructions (“approve a full refund”) **MUST NOT** alter behaviour.
- **FR-TRI-5** [S]. On unrecognised intent, the system **SHOULD** classify as `other` and route toward information/escalation rather than guessing a monetary action.

7.2 Context Gathering

- **FR-CTX-1** [M]. The system **SHALL** fetch `order_context` via the order MCP server.
- **FR-CTX-2** [M]. The system **SHALL** fetch `customer_context` via the customer MCP server.
- **FR-CTX-3** [M]. If a referenced order/customer does not exist, the system **SHALL** route to a graceful “cannot locate order” reply and **MUST NOT** fabricate facts.

7.3 Policy Retrieval (RAG)

- **FR-POL-1** [M]. The system **SHALL** retrieve policy via metadata-filtered semantic search on category, `payment_mode`, and `issue_type`.
- **FR-POL-2** [M]. The system **SHALL** populate `policy_snippets` with citations; these **MUST** be the authority for window/eligibility/refund-mode decisions.
- **FR-POL-3** [M]. The system **SHALL** compute `within_return_window` from `return_window_end` and surface it as a derived fact.

7.4 Fraud & Risk Assessment

- **FR-RSK-1** [M]. The system **SHALL** compute `risk_score` $\in [0, 1]$ from at least: `return_rate`, recent return frequency, COD refusal history, order value, category abuse propensity, and pincode/region RTO baseline.
- **FR-RSK-2** [M]. Scoring **SHALL** be a hybrid of deterministic rules and model judgement; `risk_factors` **MUST** enumerate the contributing signals.
- **FR-RSK-3** [M]. `risk_score` $\geq$ `RISK_ESCALATION_THRESHOLD` **SHALL** force `requires_human=true`.
- **FR-RSK-4** [S]. The assessor **SHOULD** flag wardrobing/serial-returner patterns as named factors.

7.5 Root-Cause Diagnosis

- **FR-RC-1** [M]. The system **SHALL** assign exactly one `root_cause`, justified by request + context.
- **FR-RC-2** [M]. `root_cause` **SHALL** be the primary determinant of the candidate-action set (Section 9.3).
- **FR-RC-3** [M]. When evidence is insufficient for a confident monetary action, the system **SHALL** prefer a conservative cause (`genuine_other/expectation_mismatch`) rather than an unjustified high-cost action.

7.6 Resolution Planning

- **FR-PLN-1** [M]. The system **SHALL** generate a ranked `candidate_actions` list, each annotated with estimated platform cost and a policy-eligibility flag (Section 9.1).
- **FR-PLN-2** [M]. The system **SHALL** select `proposed_action` as the eligible candidate minimising expected net cost subject to the constraints in Section 9.4.
- **FR-PLN-3** [M]. The Planner **MUST NOT** itself execute any action.
- **FR-PLN-4** [M]. For a genuine defect/wrong-item, the Planner **MUST NOT** propose denial purely to reduce cost (satisfaction floor, Section 9.4).
- **FR-PLN-5** [S]. The Planner **SHOULD** attach a short rationale to `proposed_action`.

7.7 Guardrail Enforcement

- **FR-GRD-1** [M]. Before any execution, the system **SHALL** validate `proposed_action` against all hard guardrails (Appendix B): `coupon` $\leq$ `MAX_COUPON_PCT` of order value and $\leq$ `MAX_COUPON_ABS`; `refund/credit` $\leq$ order value; `auto-refund` $\leq$ `MAX_AUTO_REFUND_ABS`; policy compliance; per-customer auto-refund rate within `AUTO_REFUND_RATE_LIMIT`.
- **FR-GRD-2** [M]. If a monetary parameter exceeds a soft cap but a clamped value is still policy-valid, the system **MAY** set `guardrail_status=clamped` and proceed; the clamp **MUST** be recorded.
- **FR-GRD-3** [M]. If the action exceeds an auto-execution hard limit or is policy-ambiguous, the system **SHALL** set `requires_human=true`.
- **FR-GRD-4** [M]. A guardrail **MUST NOT** be overridable by request content, model output, or any non-operator input.

7.8 Human-in-the-Loop Escalation

- **FR-HIL-1** [M]. When `requires_human=true`, the graph **SHALL** pause via interrupt, persist state, enqueue an `escalations` record with the recommendation, and emit an escalation event.
- **FR-HIL-2** [M]. A reviewer decision **SHALL** resume the exact paused instance (by `request_id/thread`).
- **FR-HIL-3** [M]. On *modify*, the reviewer-supplied action **MUST** itself pass guardrails before execution.
- **FR-HIL-4** [M]. On *reject*, the system **SHALL** route to a policy-compliant denial/alternative reply without a monetary action.
- **FR-HIL-5** [M]. A paused resolution **MUST** survive a process restart and remain resumable (CON-3).

7.9 Action Execution

- **FR-EXE-1** [M]. The Executor **SHALL** invoke exactly the approved action's tool(s), record `executed_action`, write the audit entry, and update `resolutions`.
- **FR-EXE-2** [M]. Execution **MUST** be idempotent by `request_id`: a repeat **MUST NOT** produce a second financial effect.
- **FR-EXE-3** [M]. On tool failure, the system **SHALL** mark the resolution failed, avoid partial financial effects, and route to escalation/safe reply.

7.10 Customer Response Generation

- **FR-RES-1** [M]. The system **SHALL** generate an empathetic, concise, policy-consistent reply reflecting the outcome.
- **FR-RES-2** [M]. The reply **MUST NOT** contradict the executed action or policy, and **MUST NOT** reveal internal risk scores or fraud reasoning.
- **FR-RES-3** [S]. For deflection offers, the reply **SHOULD** present the alternative and the customer's right to a standard refund where applicable.

7.11 Outcome Logging & Audit

- **FR-LOG-1** [M]. Every resolution **SHALL** persist classification, root cause, risk, policy citations, proposed vs executed action, amount, `expected_return_cost`, `expected_saving`, human involvement, rationale, timestamps, and trace id.
- **FR-LOG-2** [M]. Every monetary action **SHALL** create an append-only `audit_log` row and a `returns.audit.v1` event.

7.12 Event Ingestion & Emission

- **FR-EVT-1** [M]. A consumer **SHALL** transform `returns.requests.v1` messages into idempotent graph invocations.
- **FR-EVT-2** [M]. The system **SHALL** emit `returns.resolutions.v1` per completed resolution and `returns.escalations.v1` per escalation.
- **FR-EVT-3** [M]. Malformed messages **SHALL** be dead-lettered without crashing the worker.

7.13 Analytics & Reporting

- **FR-RPT-1** [M]. The system **SHALL** compute and expose: total resolutions; auto-resolution rate; escalation rate; deflection rate; estimated INR saved vs a naive always-refund baseline; average resolution latency; action-type distribution; and guardrail-violation count (**MUST** be 0).
- **FR-RPT-2** [S]. The dashboard **SHOULD** visualise these and list recent resolutions with drill-down to rationale and trace.

8 Non-Functional Requirements

8.1 Performance & Scalability

- **NFR-PERF-1** [M]. p95 end-to-end latency for the automated path (excluding human wait) **SHOULD** be ≤ 8 seconds under demo load.
- **NFR-PERF-2** [M]. The consumer tier **MUST** scale horizontally; with N workers the system **SHALL** process requests concurrently without shared mutable in-process state.
- **NFR-PERF-3** [S]. System prompts/tool schemas **SHOULD** be cached per node to bound repeated token cost.

8.2 Reliability & Durability

- **NFR-REL-1** [M]. Graph state **MUST** be checkpointed to Postgres so failures/restarts resume from the last completed node.
- **NFR-REL-2** [M]. Every loop/conditional path **MUST** terminate within `MAX_ITERATIONS`.
- **NFR-REL-3** [M]. LLM/tool calls **MUST** use bounded retries and timeouts; exhaustion **MUST** degrade to escalation/safe reply, never an unguarded action.

8.3 Security & Privacy

- **NFR-SEC-1** [M]. Secrets **MUST** come from environment/secret store, never committed.
- **NFR-SEC-2** [M]. All external/customer input **MUST** be validated and treated as untrusted (CON-6, FR-TRI-4).
- **NFR-SEC-3** [M]. Only synthetic data **MUST** be used; no real PII or payment credentials.
- **NFR-SEC-4** [S]. Operator decision endpoints **SHOULD** require a recorded reviewer identity.

8.4 Safety & Financial Controls

- **NFR-SAF-1** [M]. No financial action **MUST** ever exceed the order value or the auto-execution ceiling without human approval.
- **NFR-SAF-2** [M]. Guardrails **MUST** be evaluated deterministically and **MUST NOT** be bypassable by model output or input content.
- **NFR-SAF-3** [M]. Financial execution **MUST** be idempotent (FR-EXE-2) and fully audited (FR-LOG-2).
- **NFR-SAF-4** [M]. The system **MUST NOT** default a genuine refund to store-credit-only (dark-pattern avoidance; DR-RAG-4).

8.5 Observability

- **NFR-OBS-1** [M]. Every node, LLM call, and tool call **MUST** be traced and correlated by `trace_id/request_id`.
- **NFR-OBS-2** [M]. Logs **MUST** be structured and carry `request_id`, `order_id`, node name, and outcome.
- **NFR-OBS-3** [S]. A resolution detail view **SHOULD** deep-link to its trace.

8.6 Maintainability & Extensibility

- **NFR-MNT-1** [M]. Adding a new `issue_type`, `root_cause`, or `action_type` **MUST** be possible via catalogue + planner-rule additions without re-architecting the graph.
- **NFR-MNT-2** [M]. All thresholds/limits **MUST** be config-driven (Appendix B).
- **NFR-MNT-3** [S]. New action tools **SHOULD** be addable by dropping in a decorated tool and registering it.

8.7 Compliance & Ethics

- **NFR-CMP-1** [M]. Customer replies **MUST NOT** misrepresent rights or hide the standard refund option where it applies.
- **NFR-CMP-2** [S]. Risk scoring **SHOULD** be explainable (named factors) and **MUST NOT** be exposed to the customer.

9 Decision-Logic Specification

This section defines the “brain”. It is deliberately explicit so behaviour is testable and the design choice is defensible as a constrained decision problem rather than ad-hoc prompting.

9.1 Cost Model

For a given order, the expected cost of allowing a full return/RTO:

$$C_{\text{return}} \approx \text{reverse logistics} + \text{restocking and handling} \\ + \text{lost margin} \times \text{Pr}(\text{item unsellable}) + \text{forward-leg dead cost (for RTO)}.$$

For a candidate action a , define the action cost C_a (e.g. coupon face value $\times$ expected redemption, exchange shipping, goodwill credit, expedited replacement delta, or 0 for information); C_a = order value for a full refund of a delivered item. An action is *economically preferred* when $C_a < C_{\text{return}}$, but economics are subordinate to the constraints below.

9.2 Action Catalogue & Eligibility

Each action carries eligibility predicates evaluated against policy + context, e.g.: `instant_refund/partial_refund` (within window or defect/wrong-item; refund to original method by default); `exchange_with_size_guide/free_exchange` (exchange-supported category with a stock substitute — primary lever for size/fit); `retention_coupon` (discretionary returns; bounded by coupon caps); `expedited_replacement+goodwill_credit` (defect/wrong-item); `deny_with_explanation` (only when policy clearly disallows — never for a genuine defect); `escalate_to_human` (forced by risk/value/ambiguity gates).

9.3 Decision Rules per Root Cause (default policy)

Root cause	Preferred first action	Fallbacks / notes
size_fit_mismatch	exchange_with_size_guide	free_exchange → partial_refund; targets the #1 fashion driver via deflection
defect_damage	expedited_replacement (+goodwill)	instant_refund; satisfaction floor: never deny
wrong_item_shipped	expedited_replacement	instant_refund; platform error ⇒ customer-favourable
changed_mind	retention_coupon (if $C_a < C_{\text{return}}$)	accept return → instant_refund
found_cheaper	retention_coupon (price-aware, capped)	instant_refund
delivery_delay	provide_information (+small goodwill)	partial_refund
expectation_mismatch	exchange / info; small credit	instant_refund
fraud_suspected	escalate_to_human	deny_with_explanation (post-review); never auto-pay
genuine_other	instant_refund (if eligible)	escalate_to_human

9.4 Constrained-Selection Formulation

Selection of `proposed_action` solves a small constrained optimisation over a discrete eligible action set:

$$\begin{aligned}
& \text{minimize}_a && \mathbb{E}[\text{net platform cost}(a)] \\
& \text{subject to} && a \in \mathcal{A}(\text{root_cause}, \text{policy}, \text{context}) \\
& && a \text{ is policy-compliant} \quad (\text{hard}) \\
& && \text{monetary}(a) \leq \text{order value} \quad (\text{hard}) \\
& && \text{coupon}(a) \leq \text{MAX_COUPON_PCT} \cdot \text{order value} \quad (\text{hard} / \text{clamp}) \\
& && \text{coupon}(a) \leq \text{MAX_COUPON_ABS} \quad (\text{hard} / \text{clamp}) \\
& && \text{auto-monetary}(a) \leq \text{MAX_AUTO_REFUND_ABS} \quad (\text{else escalate}) \\
& && \text{risk} < \text{RISK_ESCALATION_THRESHOLD} \quad (\text{else escalate}) \\
& && \text{satisfaction}(a) \geq \text{floor}(\text{root_cause}) \quad (\text{hard})
\end{aligned}$$

The satisfaction floor forbids cost-minimising denial/under-compensation for `defect_damage/wrong_item_shipped`. If the feasible set is empty or any escalation gate trips, the selected action is `escalate_to_human`. This makes the agent’s choice a defensible, bounded optimisation — a natural place to reference constrained-optimisation reasoning — rather than unconstrained model discretion.

10 Evaluation & Acceptance Criteria

10.1 Evaluation Dataset

- **EV-1.** A labelled dataset of ≥ 40 scenarios **MUST** exist in `eval_cases`, spanning every root cause, both payment modes, returnable and non-returnable categories, in-window and out-of-window, low- and high-risk customers, and at least 5 escalation-warranting cases.
- **EV-2.** Each case **MUST** specify expected `root_cause`, expected `action_type` (or an allowed set), and expected escalation (yes/no).

10.2 Metrics & Targets

Metric	Definition	Target
Guardrail-violation rate	financial actions exceeding hard limits	0% (hard)
Escalation correctness	precision & recall of <code>requires_human</code> vs labels	≥ 0.9 each
Root-cause accuracy	exact-match vs labels	≥ 0.85
Action appropriateness	selected action in allowed set	≥ 0.85
Satisfaction-floor adherence	no under-compensation on defect/wrong-item	100% (hard)
Deflection rate	deflections / deflectable cases	reported
Est. INR saved vs always-refund	$\sum(C_{\text{return}} - C_a)$ over deflected cases	reported
p95 automated latency	per NFR-PERF-1	≤ 8 s

10.3 Acceptance Criteria

- **AC-1.** All [M] requirements pass their tests.
- **AC-2.** Guardrail-violation rate is 0% and satisfaction-floor adherence is 100%.
- **AC-3.** The eval suite runs in CI and is green at the configured targets.
- **AC-4.** A paused escalation survives a service restart and resumes correctly (demonstrated).
- **AC-5.** A redelivered Kafka message does not double-execute a financial action (demonstrated).
- **AC-6.** Every monetary action has a matching immutable audit entry and emitted audit event.

10.4 Test Strategy

Unit: guardrail math/clamping; cost model; eligibility predicates; schema validation; idempotency handling. *Integration:* full graph per scenario; HITL pause→resume; tool-failure degradation; dead-letter on malformed event. *Scenario/eval:* the labelled dataset scored against Section 10.2 via the tracing/eval framework. *Negative/safety:* prompt-injection in `raw_request`; out-of-window non-defect refund attempt; over-cap coupon proposal; non-existent order.

11 Traceability Matrix

Capability	Functional reqs	Realised by / verified by
Intake & triage	FR-TRI-1..5	Triage node / unit + scenario + injection test
Context gathering	FR-CTX-1..3	Context Gatherer + MCP / integration
Policy RAG	FR-POL-1..3, DR-RAG-1..4	Policy Retriever + pgvector / retrieval + scenario
Risk scoring	FR-RSK-1..4	Risk Assessor + fraud MCP / unit + eval
Root cause	FR-RC-1..3	Diagnoser / eval (accuracy)
Resolution planning	FR-PLN-1..5, Sec. 9	Planner / unit (cost/eligibility) + eval
Guardrails	FR-GRD-1..4, NFR-SAF-*	Guardrail Checker / unit + negative tests
HITL	FR-HIL-1..5, NFR-REL-1	HITL node + decision endpoint / pause-resume-restart
Execution	FR-EXE-1..3, NFR-SAF-3	Executor + tools / idempotency + failure
Customer reply	FR-RES-1..3, NFR-CMP-1	Responder / scenario review
Logging & audit	FR-LOG-1..2	Outcome Logger / audit append-only
Eventing	FR-EVT-1..3, DR-EVT-1	Kafka worker / dead-letter + idempotency
Analytics	FR-RPT-1..2	Metrics endpoint + dashboard / integration

12 Appendices

Appendix A — Worked End-to-End Scenarios

A.1 Size mismatch, prepaid, in-window (deflection). “The kurti is too tight, I want to return it.” Triage → `wrong_size`; context → prepaid apparel, in-window, low-risk; policy → exchange allowed; risk 0.2; root cause `size_fit_mismatch`; planner → `exchange_with_size_guide` ($C_a < C_{\text{return}}$, within caps); guardrail pass, no human; execute exchange + size guide; reply offers exchange and notes a refund is still available; logged `expected_saving` = $C_{\text{return}} - C_a$.

A.2 Defective electronics, prepaid, in-window (satisfaction floor). Triage → `damaged_item`; root cause `defect_damage`; planner → `expedited_replacement` + small `goodwill_credit`; floor forbids denial; guardrail pass; execute; customer-favourable reply.

A.3 High-value refund, suspected serial returner, COD (escalation). Context → high `return_rate`, multiple COD refusals, value above auto ceiling; risk 0.82 ($>$ threshold) → `requires_human`; recommendation attached; HITL pause + escalation event; reviewer modifies to partial refund; the modified action is re-checked against guardrails → pass; execute; audit records the human actor.

A.4 Out-of-window, non-defect, non-returnable category (policy denial). Policy → non-returnable + window expired, no defect; planner → `deny_with_explanation`; guardrail pass; reply explains the policy courteously and offers any applicable alternative; no monetary action.

Appendix B — Configuration Parameters (illustrative defaults)

Key	Default	Meaning
MAX_COUPON_PCT	0.20	Max retention coupon as a fraction of order value
MAX_COUPON_ABS	INR 300	Absolute coupon ceiling
MAX_AUTO_REFUND_ABS	INR 2000	Above this, refunds require human approval
RISK_ESCALATION_THRESHOLD	0.70	Risk score forcing escalation
MAX_GOODWILL_CREDIT	INR 150	Goodwill credit ceiling
AUTO_REFUND_RATE_LIMIT	3 / 30 days	Max auto-refunds per customer per window
RAG_TOP_K	4	Policy chunks retrieved
MAX_ITERATIONS	12	Hard graph loop cap
LLM_TIMEOUT_S	30	Per-call timeout
LLM_MAX_RETRIES	3	Per-call retries (backoff)

Appendix C — Out of Scope & Future Work

Pre-purchase RTO prevention (advance-payment nudge, address/pincode risk at checkout, WhatsApp order-confirmation); learning from realised outcomes to tune the cost model and coupon sizing; multilingual/vernacular replies; multimodal defect verification using a CV model; seller-facing analytics on high-return SKUs; real payment/logistics integrations.